# Benchmarking YOLO and DETR-Based Architectures for Object Detection on Small-Object-Biased Public Datasets

Oren Elashri
Practical Work in AI, 344.031, Winter Semester 2025/26
Silicon Austria Labs
oren.elashri@silicon-austria.com
April 20, 2026

## 1 Introduction

**Over the past decade, object detection has moved from handcrafted pipelines to end-to-end deep architectures**, fundamentally changing how localization and classification are coupled. Early methods relied on engineered features and sliding-window classifiers, but the advent of convolutional neural networks (CNNs) led to region-based approaches such as R-CNN and Faster R-CNN, which combine deep feature extraction with region proposal mechanisms [1, 2].

More recently, single-stage detectors and transformer-based architectures have significantly reshaped the field. Single-stage detectors such as the YOLO family perform object localization and classification in a single forward pass, enabling real-time object detection [3].Meanwhile, transformer-based models such as DETR reformulate object detection as a direct set prediction problem using attention mechanisms that allow global reasoning across the entire image [4, 5].

Despite these advances, **detecting small objects continues to be a significant challenge within computer vision**. Small objects occupy only a limited number of pixels within an image, making them particularly sensitive to localization errors, background clutter, and feature downsampling within deep neural networks. As a result, **detection performance for small objects** is often substantially **lower than for medium and large objects** [6].

Small object detection is highly relevant for many real-world applications including autonomous driving, aerial surveillance, traffic monitoring, and robotics. In such scenarios, small objects may represent critical targets such as distant pedestrians, traffic lights, or small vehicles. Improving the reliability of small object detection is therefore an important research objective.

### 1.1 Motivation

Single-stage detectors from the **YOLO family have become dominant in real-time object detection** due to their favorable balance between accuracy and inference speed [3, 7]. Recent YOLO variants **incorporate architectural improvements** such as enhanced feature aggregation and attention mechanisms **to improve detection performance across object scales**.

At the same time, transformer-based architectures such as DETR introduce a fundamentally different detection paradigm. Instead of relying on dense grid predictions, **DETR models predict a fixed set of object queries using attention mechanisms** that capture global contextual information [4].

Although transformer-based detectors demonstrate strong performance on general detection benchmarks, their **computational cost and effectiveness for small object detection remain open questions**. In particular, it is unclear whether the global reasoning capabilities of transformers provide advantages when detecting small objects that occupy only a small portion of the image [4, 8].

Furthermore, several benchmarking studies focus on a single dataset, making it difficult to assess whether models generalize across different data distributions [9]. Dataset bias may significantly affect detection performance, especially for small objects [10].

### 1.2 Research Questions

This work addresses the following research questions:

1. **Which model (YOLO-based vs. DETR-based architectures) achieves the best trade-off in accuracy (mAP and $AP_{small}$), latency, and parameter count when evaluated on small-object-focused subsets of MS COCO and OpenImages V7?**
2. **How does dataset bias, particularly the differences in small-object frequency and scale distribution between MS COCO 2017 and OpenImages V7, affect small-object detection performance (accuracy, latency, and cross-dataset generalization)?**
3. **Does increasing model capacity (parameter count) consistently improve accuracy and the accuracy–latency trade-off for small object detection?**

### 1.3 Related Work

**Object detection** approaches are typically **divided into two-stage and one-stage detectors**.

**Two-stage detectors** such as Faster R-CNN **first generate** candidate object **proposals and subsequently perform classification and bounding box refinement**. While this design often yields good localization accuracy, it **introduces additional computational** overhead and **slower inference speed** compared to single-stage methods [2].

**One-stage detectors** eliminate the proposal stage and **directly predict bounding boxes and class labels in a single forward pass**. The YOLO family represents one of the most widely adopted single-stage architectures due to its strong balance between accuracy and real-time performance [3].

Instead of predicting dense bounding boxes on feature maps, **DETR models treat object detection as a set prediction problem.** A fixed number of learnable object queries are processed by a transformer decoder that outputs object predictions. These predictions are matched to ground truth annotations using bipartite matching during training [4].

While the transformer architecture enables global reasoning through self-attention, early DETR models exhibited slow convergence and limited performance on small objects. More recent models such as **RT-DETR introduce hybrid CNN-transformer architectures** (e.g. CCFF), that substantially improve inference efficiency and detection accuracy [8].

While these approaches demonstrate strong performance on standard benchmarks, their behavior under small-object-dominated conditions remains an area with room for further exploration. In particular, prior work rarely examined the interaction between architectural design choices and object scale distribution. This gap motivates the comparative analysis conducted in the present study. **Small-object detection is fundamentally difficult**, as downsampling reduces spatial detail and leaves little information for the model to work with. This study offers clear insights into how modern convolutional and transformer-based detectors perform under small-object conditions and across different datasets.

### 1.4 Outline

The remainder of this report is structured as follows. Chapter 2 presents the **material and methods**, including a detailed analysis of the related work, two datasets, the small-object definition, and the evaluated architectures (YOLOv5u, YOLO11, and RT-DETR). Chapter 3 describes the **implementation** of the reproducible benchmarking framework and highlights selected key code components. Chapter 4 first explains the evaluation measures and then presents the **experimental results** structured by the three research questions in Section 1.2. Chapter 5 **discusses the findings, limitations, and future research directions**. Finally, the references are listed.

## 2 Material and Methods

### 2.1 Small Object Definition

In this work, small objects are defined according to the official MS COCO scale definition [11]. Using a consistent definition across datasets ensures comparability and enables direct use of the COCO evaluation protocol [12].

The COCO scale categories are defined as:

- Small: area $< 32^2$ pixels
- Medium: $32^2 \leq$ area $< 96^2$ pixels
- Large: area $\geq 96^2$ pixels

This definition is **applied uniformly** to **both COCO and OpenImages**, **allowing consistent small-object analysis and cross-dataset evaluation**.

### 2.2 Datasets

Two publicly available datasets were selected for benchmarking: **MS COCO 2017** [11] and **OpenImages V7** [13]. These datasets provide complementary distributions of object scales and class frequencies, enabling evaluation of model robustness under different data conditions.

| Dataset | Total objects | Small objects | Small object ratio |
|---|---|---|---|
| COCO | 896,782 | 279,170 | 31.13% |
| OpenImages | 14,914,188 | 1,996,747 | 13.39% |

Table 1: Small-object prevalence in COCO vs OpenImages, illustrating substantial dataset bias.

#### 2.2.1 MS COCO 2017

MS COCO [11] is one of the most widely used object detection datasets. It contains approximately:

- 330,000 images
- 80 object categories
- 897,000 annotated bounding boxes

These standardized scale definition introduced makes **COCO** particularly **suitable for benchmarking small-object detection performance.**

Table 2 and Figures 1–2 summarise the 30 COCO classes that exhibit the highest small-object prevalence (i.e., the highest percentage of instances with area $< 32^2$ pixels). Classes such as *sports ball* and *traffic light* show more than 70% small objects. Together, these analyses motivated the selection of this COCO-specific Top-30 subset (ranked by small-object percentage, all $\geq 30\%$) as a focused yet representative benchmark for small-object detection.

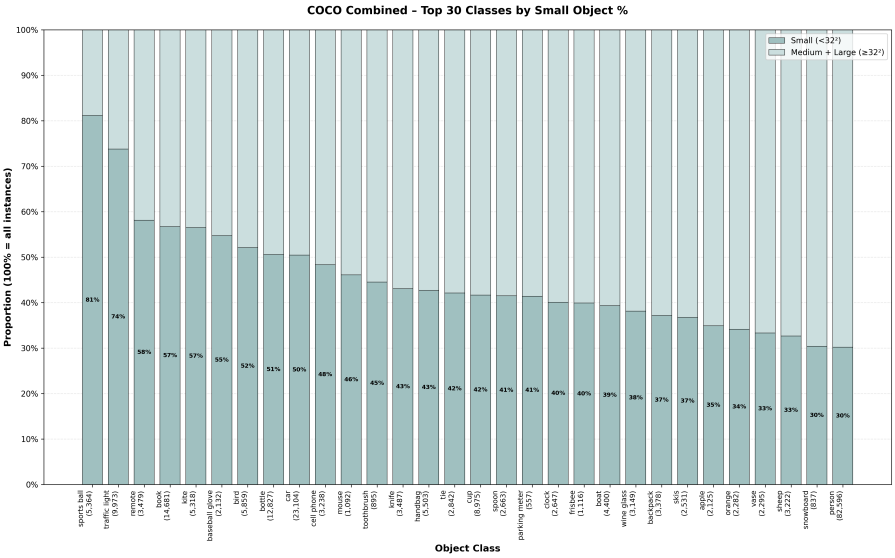


Figure 1: Normalized small-object ratio across the 30 COCO classes with the highest small-object prevalence (COCO train + val combined dataset). Classes such as *sports ball* and *traffic light* show the highest relative prevalence, as discussed in Section 2.2.1.

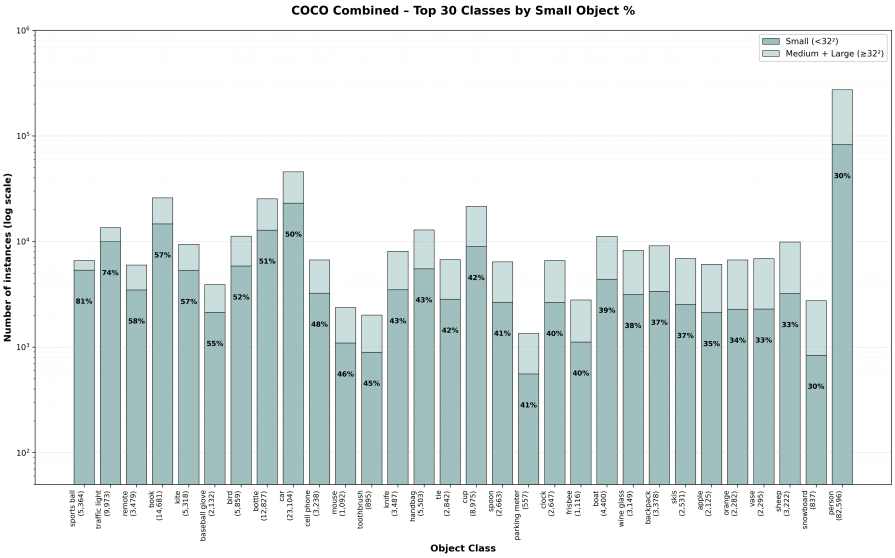


Figure 2: Log-scale absolute small-object instance counts for the 30 COCO classes with the highest small-object prevalence (COCO train + val combined dataset). Despite lower relative ratios, high-frequency classes such as *person* and *car* dominate the training signal (i.e., they provide the largest number of training examples for the model).

| COCO TRAIN + VAL - TOP 30 BY SMALL OBJECT % ($< 32^2$ px) | | | | |
|---|---|---|---|---|
| Rank | Class | Total | Small | Small % |
| 1 | sports ball | 6,610 | 5,364 | 81.15% |
| 2 | traffic light | 13,521 | 9,973 | 73.76% |
| 3 | remote | 5,986 | 3,479 | 58.12% |
| 4 | book | 25,876 | 14,681 | 56.74% |
| 5 | kite | 9,412 | 5,318 | 56.50% |
| 6 | baseball glove | 3,895 | 2,132 | 54.74% |
| 7 | bird | 11,246 | 5,859 | 52.10% |
| 8 | bottle | 25,367 | 12,827 | 50.57% |
| 9 | car | 45,799 | 23,104 | 50.45% |
| 10 | cell phone | 6,696 | 3,238 | 48.36% |
| 11 | mouse | 2,368 | 1,092 | 46.11% |
| 12 | toothbrush | 2,011 | 895 | 44.51% |
| 13 | knife | 8,096 | 3,487 | 43.07% |
| 14 | handbag | 12,894 | 5,503 | 42.68% |
| 15 | tie | 6,750 | 2,842 | 42.10% |
| 16 | cup | 21,549 | 8,975 | 41.65% |
| 17 | spoon | 6,418 | 2,663 | 41.49% |
| 18 | parking meter | 1,345 | 557 | 41.41% |
| 19 | clock | 6,601 | 2,647 | 40.10% |
| 20 | frisbee | 2,797 | 1,116 | 39.90% |
| 21 | boat | 11,189 | 4,400 | 39.32% |
| 22 | wine glass | 8,256 | 3,149 | 38.14% |
| 23 | backpack | 9,091 | 3,378 | 37.16% |
| 24 | skis | 6,887 | 2,531 | 36.75% |
| 25 | apple | 6,090 | 2,125 | 34.89% |
| 26 | orange | 6,686 | 2,282 | 34.13% |
| 27 | vase | 6,890 | 2,295 | 33.31% |
| 28 | sheep | 9,870 | 3,222 | 32.64% |
| 29 | snowboard | 2,754 | 837 | 30.39% |
| 30 | person | 273,469 | 82,596 | 30.20% |

Table 2: Statistics of small objects (area $< 32^2$ pixels) for the 30 COCO classes with the highest small-object prevalence, ranked by descending small-object percentage.

#### 2.2.2 OpenImages V7

OpenImages V7 [13] is a large-scale object detection dataset containing:

- nearly 1.8 million images
- 600 object categories
- approximately 15 million bounding box annotations

Compared to COCO, OpenImages contains a **significantly larger number of images** and **exhibits greater variation** in scene composition and object scale.

Table 3, and Figures 3–4 summarize the Top-30 OpenImages classes ranked by small-object frequency. It highlights categories such as *human eye* and *construction nail*, none of which exceed a 70% small-object rate.

Importantly, 24 of the Top-30 COCO small-object classes also appear in OpenImages, enabling cross-dataset comparison of scale distributions.

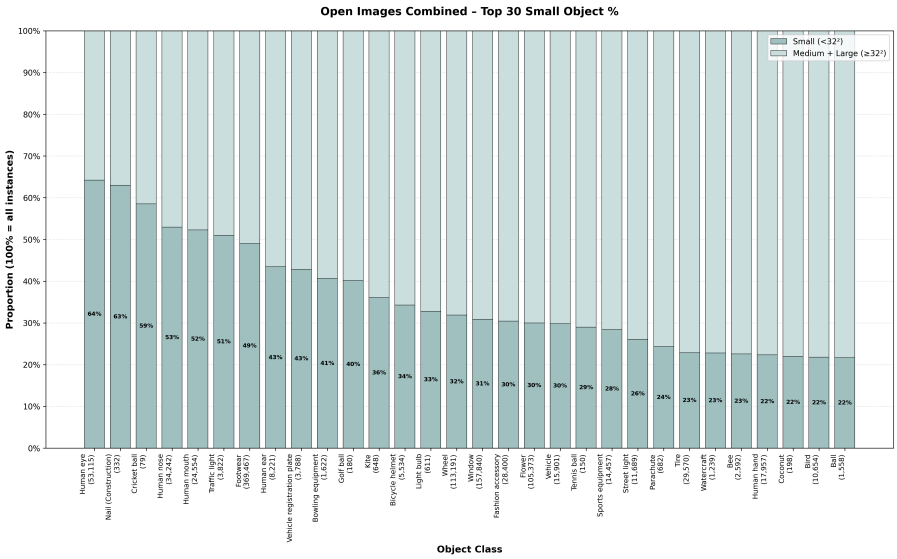


Figure 3: Normalized small-object ratio across the 30 OpenImages V7 classes with the highest small-object prevalence (OpenImages train + val combined dataset). The distribution is notably flatter than in COCO, with no class exceeding 70% small-object prevalence.

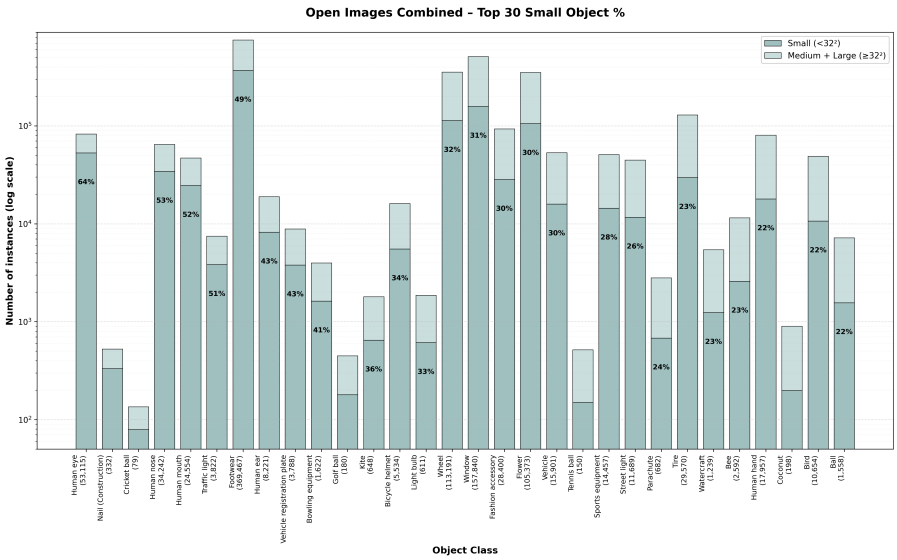


Figure 4: Log-scale absolute small-object counts for the 30 OpenImages V7 classes with the highest small-object prevalence (OpenImages train + val combined dataset). A small number of very high-frequency categories such as *Footwear, Window, Wheel* and *Flower* account for a disproportionate share of all small instances.

| OPEN IMAGES V7 TRAIN + VAL - TOP 30 BY SMALL OBJECT % ($< 32^2$ px) | | | | |
|---|---|---|---|---|
| Rank | Class | Total | Small | Small % |
| 1 | Human eye | 82,683 | 53,115 | 64.24% |
| 2 | Nail (Construction) | 527 | 332 | 63.00% |
| 3 | Cricket ball | 135 | 79 | 58.52% |
| 4 | Human nose | 64,641 | 34,242 | 52.97% |
| 5 | Human mouth | 46,949 | 24,554 | 52.30% |
| 6 | Traffic light | 7,491 | 3,822 | 51.02% |
| 7 | Footwear | 753,584 | 369,467 | 49.03% |
| 8 | Human ear | 18,921 | 8,221 | 43.45% |
| 9 | Vehicle registration plate | 8,839 | 3,788 | 42.86% |
| 10 | Bowling equipment | 3,991 | 1,622 | 40.64% |
| 11 | Golf ball | 448 | 180 | 40.18% |
| 12 | Kite | 1,796 | 648 | 36.08% |
| 13 | Bicycle helmet | 16,139 | 5,534 | 34.29% |
| 14 | Light bulb | 1,862 | 611 | 32.81% |
| 15 | Wheel | 355,022 | 113,191 | 31.88% |
| 16 | Window | 511,050 | 157,840 | 30.89% |
| 17 | Fashion accessory | 93,327 | 28,400 | 30.43% |
| 18 | Flower | 351,119 | 105,373 | 30.01% |
| 19 | Vehicle | 53,347 | 15,901 | 29.81% |
| 20 | Tennis ball | 517 | 150 | 29.01% |
| 21 | Sports equipment | 50,912 | 14,457 | 28.40% |
| 22 | Street light | 44,792 | 11,689 | 26.10% |
| 23 | Parachute | 2,801 | 682 | 24.35% |
| 24 | Tire | 129,357 | 29,570 | 22.86% |
| 25 | Watercraft | 5,431 | 1,239 | 22.81% |
| 26 | Bee | 11,491 | 2,592 | 22.56% |
| 27 | Human hand | 80,338 | 17,957 | 22.35% |
| 28 | Coconut | 901 | 198 | 21.98% |
| 29 | Bird | 48,823 | 10,654 | 21.82% |
| 30 | Ball | 7,188 | 1,558 | 21.68% |

Table 3: Statistics of small objects (area $< 32^2$ pixels) for the 30 OpenImages V7 classes with the highest small-object prevalence, ranked by descending small-object percentage.

The differences between the two datasets become clearer when comparing matched classes directly. Table 4 and Figures 5–6 report small-object statistics for the 24 classes that appear in both datasets.

The comparison shows that **COCO consistently exhibits higher small-object ratios** across most categories. In contrast, **OpenImages tends to contain larger object instances** even when the same class labels are present.

Together, these analyses demonstrate that COCO and OpenImages impose fundamentally different small-object priors. As a result, evaluating detection models on both datasets provides a stronger test of generalization and robustness.

| COCO VS OPEN IMAGES - COMPARISON TABLE (24 Matched Classes) | | | | | | | |
|---|---|---|---|---|---|---|---|
| Rank | Class | COCO Total | COCO Small | COCO Small % | OI Total | OI Small | OI Small % |
| 1 | traffic light | 13,521 | 9,973 | 73.76% | 7,491 | 3,822 | 51.02% |
| 2 | book | 25,876 | 14,681 | 56.74% | 42,000 | 2,412 | 5.74% |
| 3 | kite | 9,412 | 5,318 | 56.50% | 1,796 | 648 | 36.08% |
| 4 | baseball glove | 3,895 | 2,132 | 54.74% | 2,566 | 443 | 17.26% |
| 5 | bird | 11,246 | 5,859 | 52.10% | 48,823 | 10,654 | 21.82% |
| 6 | bottle | 25,367 | 12,827 | 50.57% | 40,686 | 1,864 | 4.58% |
| 7 | car | 45,799 | 23,104 | 50.45% | 257,999 | 24,668 | 9.56% |
| 8 | mouse | 2,368 | 1,092 | 46.11% | 912 | 5 | 0.55% |
| 9 | toothbrush | 2,011 | 895 | 44.51% | 235 | 1 | 0.43% |
| 10 | knife | 8,096 | 3,487 | 43.07% | 930 | 1 | 0.11% |
| 11 | handbag | 12,894 | 5,503 | 42.68% | 2,587 | 6 | 0.23% |
| 12 | tie | 6,750 | 2,842 | 42.10% | 10,660 | 203 | 1.90% |
| 13 | spoon | 6,418 | 2,663 | 41.49% | 1,782 | 16 | 0.90% |
| 14 | parking meter | 1,345 | 557 | 41.41% | 213 | 0 | 0.00% |
| 15 | clock | 6,601 | 2,647 | 40.10% | 1,246 | 19 | 1.52% |
| 16 | boat | 11,189 | 4,400 | 39.32% | 80,112 | 13,399 | 16.73% |
| 17 | wine glass | 8,256 | 3,149 | 38.14% | 13,084 | 265 | 2.03% |
| 18 | backpack | 9,091 | 3,378 | 37.16% | 1,248 | 21 | 1.68% |
| 19 | apple | 6,090 | 2,125 | 34.89% | 4,026 | 381 | 9.46% |
| 20 | orange | 6,686 | 2,282 | 34.13% | 6,371 | 879 | 13.80% |
| 21 | vase | 6,890 | 2,295 | 33.31% | 2,568 | 16 | 0.62% |
| 22 | sheep | 9,870 | 3,222 | 32.64% | 3,560 | 543 | 15.25% |
| 23 | snowboard | 2,754 | 837 | 30.39% | 967 | 68 | 7.03% |
| 24 | person | 273,469 | 82,596 | 30.20% | 1,051,472 | 113,627 | 10.81% |

Table 4: Small-object statistics for the 24 classes shared between MS COCO and OpenImages V7. *COCO Small %* and *OI Small %* denote the percentage of instances with area $< 32^2$ pixels *within that class* in the respective dataset.

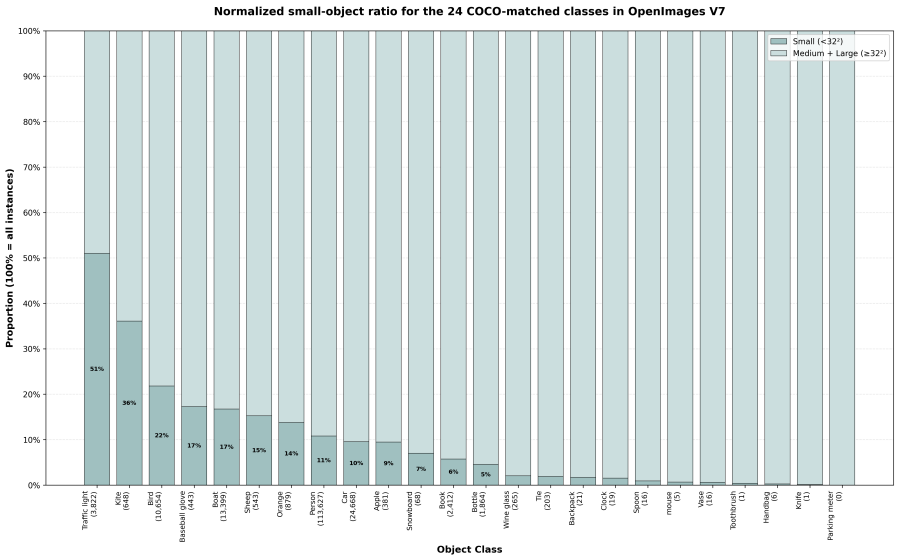


Figure 5: illustrates the **normalized small-object ratio for the 24 COCO-matched classes in OpenImages V7**, highlighting relative scale bias independent of class frequency. OpenImages exhibits a flat distribution with significantly reduced small-object ratios across most categories. This normalized view isolates scale effects and confirms that the observed differences are not merely driven by dataset size.

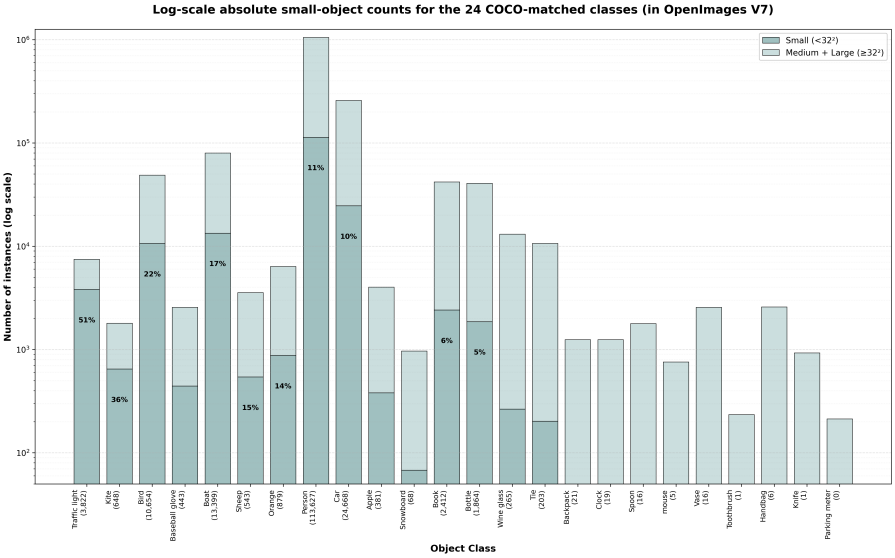


Figure 6: shows a **log-scale frequency plot of absolute small-object counts for the 24 COCO-matched classes in OpenImages V7**. While COCO emphasizes high small-object ratios, OpenImages dominates in absolute instance counts for high-frequency categories such as person, car, and boat. This imbalance highlights a critical trade-off between relative scale bias and absolute data volume, both of which influence detector behavior.

### 2.3 Detection Architectures

Three modern object detection architectures were evaluated: YOLOv5u, YOLO11, and RT-DETR.

#### 2.3.1 YOLO Architecture

YOLO detectors follow a three-stage architecture consisting of:

1. **Backbone** for feature extraction,
2. **Neck** for multi-scale feature aggregation (PANet),
3. **Detection head** simultaneous bounding-box regression and class-probability prediction.

Both YOLOv5u and YOLO11 employ **anchor-free detection heads**. Instead of predicting predefined anchor boxes, the head directly regresses bounding-box offsets together with a vector of class probabilities for each predicted box. This design enables more flexible regression across object scales [14].

YOLO11 additionally introduces attention-based modules (C2PSA blocks) that first appeared in YOLOv10 and were expanded in YOLO11 [15, 16, 7].

#### 2.3.2 DETR and RT-DETR

DETR-based detectors employ a transformer architecture to frame object detection as a set prediction problem, replacing conventional hand-designed detection heads with a fixed-size set of object queries that directly predict bounding boxes and class labels. A CNN backbone extracts feature maps processed by a transformer encoder–decoder, which performs global reasoning through attention mechanisms while shifting the burden of localization from dense spatial priors to learned query representations. In the context of small-object detection, this raises the question of whether global context **sufficiently compensates** for the reduced emphasis on fine-grained spatial detail. RT-DETR refines the transformer paradigm through hybrid CNN–transformer backbones with cross-scale feature fusion (CCFF), multi-scale feature encoders, reduced query counts, and optimised attention operations. These improvements substantially enhance inference efficiency and detection accuracy while enabling real-time deployment [4, 8].

# 3 Implementation

The benchmarking framework was implemented in PyTorch using the Ultralytics library for all models [14, 15, 17].

## 3.1 Experimental Setup

Experiments ran on a shared high-performance computing cluster (NVIDIA A100, 80GB VRAM) via SLURM, with one GPU node per benchmark. Key settings: batch size 16, input 640×640, 80 warm-up batches, and 150 timed batches. All models (YOLO and RT-DETR) used FP16 inference.

## 3.2 Reproducibility

To ensure reproducibility:

- All outputs were stored as JSON files
- Evaluation results were automatically logged and visualized
- Timestamped experiment runs were recorded

This design enables independent verification and reproducibility of all reported results.

## 3.3 Key Code Components

The complete benchmarking and analysis pipeline is implemented in three Python scripts:

1. **Dataset & Models Downloader.** Automates the download and preparation of the MS COCO 2017, OpenImages V7 datasets, and the model weights needed.
2. **Dataset Analysis.** Computes small-object statistics and generates the Top-30 class tables (based on small-object percentages in COCO and OpenImages V7) and the corresponding visualization figures presented in Section 2.2. This script performs large-scale annotation analysis to identify classes with high small-object prevalence.
3. **Benchmark Evaluation.** Implements the evaluation system described in Section 3.4 and performs model benchmarking, latency measurement, and metric computation.

## 3.4 Evaluation System Architecture

The benchmarking framework centers on an evaluation system designed to measure detection accuracy and inference efficiency in a controlled and reproducible manner. It places particular emphasis on the performance of small-object detection.

The evaluation pipeline consists of four main modules:

1. **Data Loader.** Loads images and annotations from the MS COCO 2017 and OpenImages V7 datasets. Preprocessing includes resizing, normalization, and batching to ensure consistent inputs across models.
2. **Model Module.** Loads the evaluated architectures (YOLOv5u, YOLO11, and RT-DETR) using official pretrained weights. Models run in inference mode with optional mixed-precision (FP16) to improve efficiency.
3. **Inference Engine.** Executes inference under fixed experimental conditions, including consistent input resolution and batch size. GPU synchronization is used during latency measurement to obtain reliable runtime statistics.
4. **Evaluation Module.** Converts predictions to COCO-compatible JSON format and computes metrics using the official COCO evaluation toolkit (`COCOeval`) [11, 12]. A filtered protocol is additionally applied to compute metrics for the 24 classes shared between COCO and OpenImages.

This architecture ensures that all models are evaluated under identical conditions, enabling fair comparison of detection accuracy, latency, and small-object performance across datasets. The following excerpts illustrate key implementation components of the benchmarking pipeline.

**Listing 1: Latency measurement (FPS → ms conversion)**
All latency measurements follow the standard PyTorch benchmarking approach - warm-up batches, explicit GPU synchronization via `torch.cuda.synchronize()`, and median timing across 150 batches - applied consistently to every model on both datasets (results reported in Tables 5–7) [18].

```
def measure_fps_yolo(model, loader, warmup_batches=80,
timing_batches=150):
    """Measure FPS for YOLO models with FP16"""
    model.model.eval()
    # Warmup
    with torch.inference_mode():
        for i, (img_batch, _, _) in enumerate(loader):
            if i >= warmup_batches: break
            _ = model(img_batch.to(DEVICE), verbose=False, half=True,
            imgsz=640)
    torch.cuda.synchronize()
    # Timing
    times = []
    with torch.inference_mode():
        for i, (img_batch, _, _) in enumerate(loader):
            if i >= timing_batches: break
            torch.cuda.synchronize()
            start = time.time()
            _ = model(img_batch.to(DEVICE), verbose=False, half=True,
            imgsz=640)
            torch.cuda.synchronize()
            elapsed = time.time() - start
            times.extend([elapsed / img_batch.shape[0]] *
            img_batch.shape[0])
    avg_time = np.median(times)
    return round(1 / avg_time if avg_time > 0 else 0.0, 2)
```

A nearly identical function (`measure_fps_rtdetr`) handles RT-DETR models.

**Listing 2: Hybrid COCO evaluation for YOLO models** The benchmarking script combines Ultralytics' efficient `model.val()` implementation with a custom filtered `COCOeval` stage from the PyTorch COCO API [12]. This approach enables standard COCO metrics while additionally computing $\mathrm{AP}_{\mathrm{small}}$ for the subset of 24 classes shared between COCO and OpenImages.

```
metrics = model.val(
    data=COCO_YAML, imgsz=640, batch=BATCH_SIZE, half=True,
    save_json=True, verbose=False
)
#  Custom filtered COCOeval for the 24 matched classes
if os.path.exists(pred_path):
    coco_gt = COCO(COCO_ANN)
    matched_cat_ids = [coco_name_to_id[cls] for cls in MATCHED_CLASSES]
    matched_img_ids = ...  # extracted from ground truth
    coco_dt = coco_gt.loadRes(pred_path)
```

```
    coco_eval = COCOeval(coco_gt, coco_dt, "bbox")
    coco_eval.params.catIds = matched_cat_ids
    coco_eval.params.imgIds = matched_img_ids
    coco_eval.evaluate()
    coco_eval.accumulate()
    ap_small = coco_eval.stats[3]    # ← AP_small used in all tables
```

**Listing 3: Dataset analysis pipeline – small-object statistics and cross-dataset comparison** The dataset analysis script computes small-object statistics across both datasets. The following excerpt illustrates the vectorized counting procedure used to compute per-class small-object frequencies in OpenImages. The results are used to generate the Top-30 tables (Tables 2 and 3) and the cross-dataset comparison table (Table 4).

```
def count_small_objects_oi(detections, img_sizes, area_threshold,
filter_classes=None):
    """Vectorised small-object counting for OpenImages (with mouse
    renaming)"""
    detections = detections.copy()
    detections['DisplayName'] =
    detections['DisplayName'].replace('Computer mouse', 'mouse')
    if filter_classes:
        filter_set = {fc.lower() for fc in filter_classes}
        detections = detections[detections['DisplayName'].str.lower().i
        sin(filter_set)]

    # Vectorised bbox area calculation
    detections['width'] = detections['ImageID'].map(lambda x:
    img_sizes.get(x, (0,0))[0])
    detections['bbox_w'] = (detections['XMax'] - detections['XMin']) *
    detections['width']
    detections['area'] = detections['bbox_w'] * detections['bbox_h']

    grouped = detections.groupby('DisplayName').agg(
        total=('area', 'size'),
        small=('area', lambda x: (x < area_threshold).sum())
    ).reset_index()
    # returns per_class_total and per_class_small
```

The full scripts, plots, and reproducibility instructions are provided in the accompanying directory.

## 4 Experimental Results

This section presents benchmarking results for all evaluated models on both datasets. Performance is analyzed in terms of detection accuracy, computational efficiency, and cross-dataset generalization.

### 4.1 Evaluation Metrics

Model performance was evaluated using the official COCO evaluation protocol [11]. The key measures are: (i) **Mean Average Precision (mAP)**: Average precision across IoU thresholds 0.50–0.95 ($\mathrm{AP}_{\mathrm{all}}$) [11]; (ii)$\mathbf{AP}_{\mathbf{small}}$: Average precision for objects with area $< 32^2$ pixels (the primary metric for small-object performance) [11]; (iii) $\mathbf{AP50}_{\mathrm{small}}$: AP at IoU = 0.50 for small objects [11]; (iv) **Latency**: Measured in milliseconds per image (ms/img) using FP16. All latency values are the

median over 150 timed batches after 80 warmup batches; (v) **Parameter Count**: Total number of trainable parameters in millions (M), obtained directly from the model.

These metrics enable simultaneous assessment of detection accuracy, inference speed, and model complexity. Subsequent subsections present the results structured by dataset and research question.

### 4.2 Performance on MS COCO

The selected YOLOv5u, YOLO11 and RT-DETR models are pretrained on the MS COCO 2017 training set. Their inference results on the COCO validation set are summarized in Table 5. Figure 7 presents the trade-off between detection accuracy and inference latency. A clear scaling trend can be observed within the **YOLOv5u model family: increasing model size consistently improves $\mathbf{AP_{small}}$ while moderately increasing inference latency.** The **YOLO11 models outperform their YOLOv5u** counterparts at comparable parameter counts, indicating that the architectural refinements introduced in YOLO11 improve feature aggregation and contextual modeling. The best performing model on COCO is YOLO11x, achieving an $\mathrm{AP}_{\mathrm{small}}$ of 0.3527 with an inference latency of 4.41 ms per image. The higher latency of RT-DETR (7.13–10.68 ms vs. YOLO11x at 4.41 ms) comes directly from the computational overhead of the transformer attention layers. The hybrid CCFF optimisation helps, but the attention cost is still the main reason for the slower speed [8].

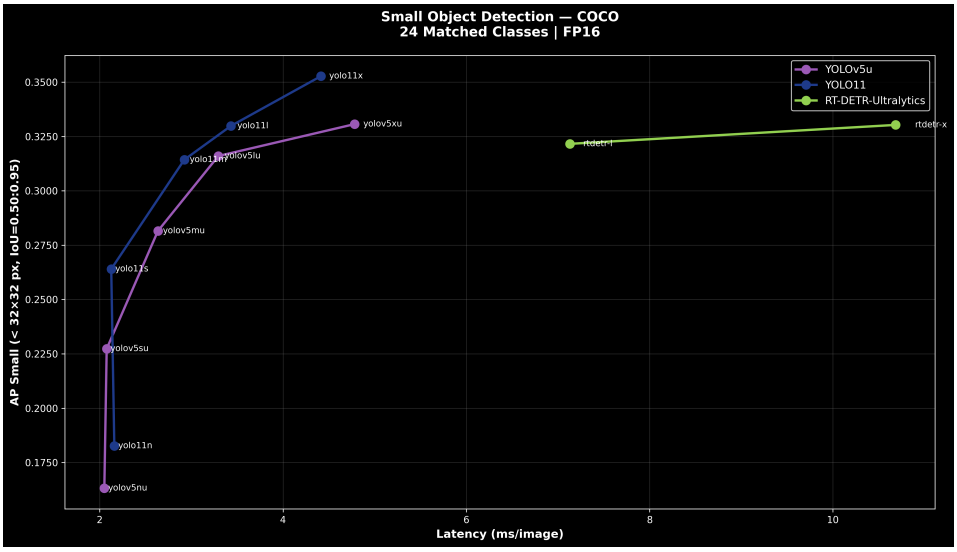


Figure 7: Performance of all evaluated models on the MS COCO validation set.

| Model | Params (M) | Latency (ms) | $\mathbf{AP}_{\mathrm{small}}$ | $\mathrm{AP}_{\mathrm{all}}$ | $\mathrm{AP50}_{\mathrm{small}}$ |
|---|---|---|---|---|---|
| yolov5nu | 2.65 | 2.05 | 0.163164 | 0.3406 | 0.2025 |
| yolov5su | 9.15 | 2.08 | 0.227325 | 0.4288 | 0.2432 |
| yolov5mu | 25.11 | 2.64 | 0.281554 | 0.4886 | 0.2665 |
| yolov5lu | 53.23 | 3.29 | 0.316002 | 0.5223 | 0.2807 |
| yolov5xu | 97.28 | 4.78 | **0.330654** | 0.5330 | 0.2873 |
| yolo11n | 2.62 | 2.16 | 0.182605 | 0.3919 | 0.2183 |
| yolo11s | 9.46 | 2.13 | 0.264008 | 0.4678 | 0.2554 |
| yolo11m | 20.11 | 2.92 | 0.314290 | 0.5148 | 0.2793 |
| yolo11l | 25.37 | 3.43 | 0.329850 | 0.5335 | 0.2869 |
| yolo11x | 56.97 | 4.41 | **0.352730** | 0.5480 | 0.2914 |
| rtdetr-l | 32.97 | 7.13 | 0.321591 | 0.5207 | 0.2765 |
| rtdetr-x | 67.47 | 10.68 | **0.330294** | 0.5373 | 0.2819 |

Table 5: Performance of YOLOv5u, YOLO11, and RT-DETR on the MS COCO validation set (COCO-pretrained weights, 640×640 input, FP16 inference). Bold values mark the best $\mathrm{AP}_{\mathrm{small}}$ per model family.

### 4.3 Performance on OpenImages V7

Performance results on the OpenImages V7 validation set are presented in Table 6 and visualised in Figure 8. The same models, pretrained exclusively on the MS COCO 2017 training set (with no fine-tuning on OpenImages), are used for evaluation. Compared to the COCO benchmark, **all models exhibit a substantial drop in $\mathrm{AP}_{\mathrm{small}}$.**

Compared to the COCO benchmark, all models exhibit a substantial drop in $\mathrm{AP}_{\mathrm{small}}$. This drop reflects a **domain shift** between the two datasets: although COCO contains a higher relative proportion of small objects (31.13% vs. 13.39% in OpenImages), the datasets also differ markedly in absolute scale distributions within shared classes, scene composition, annotation protocols, image resolutions, and class vocabulary (80 categories in COCO versus 600 in OpenImages V7). Models pretrained exclusively on COCO statistics are therefore evaluated under this distribution shift, which is known to degrade cross-dataset generalization in object detection [10]. The RT-DETRx model achieves the highest $\mathrm{AP}_{\mathrm{small}}$ score on OpenImages. However, these gains come at the cost of significantly higher inference latency compared to YOLO-based architectures.

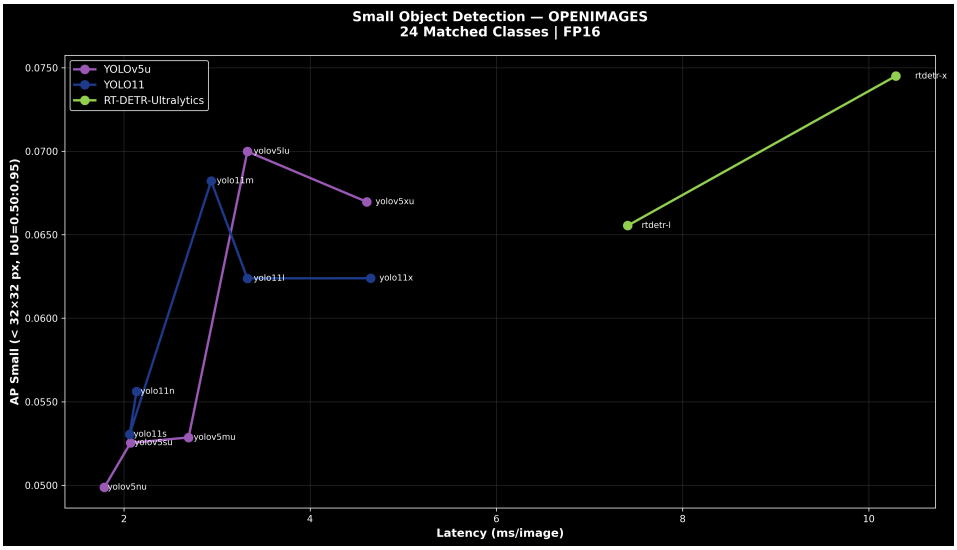


Figure 8: Performance of all evaluated models on the OpenImages V7 validation set.

| Model | Params (M) | Latency (ms) | $\mathbf{AP}_{\mathrm{small}}$ | $\mathrm{AP}_{\mathrm{all}}$ | $\mathrm{AP50}_{\mathrm{small}}$ |
|---|---|---|---|---|---|
| yolov5nu | 2.65 | 1.79 | 0.049866 | 0.1752 | 0.2372 |
| yolov5su | 9.15 | 2.07 | 0.052532 | 0.2407 | 0.2842 |
| yolov5mu | 25.11 | 2.70 | 0.052859 | 0.3251 | 0.3722 |
| yolov5lu | 53.23 | 3.33 | **0.069986** | 0.3582 | 0.3950 |
| yolov5xu | 97.28 | 4.61 | 0.066969 | 0.3808 | 0.4016 |
| yolo11n | 2.62 | 2.14 | 0.055607 | 0.2433 | 0.3034 |
| yolo11s | 9.46 | 2.06 | 0.053039 | 0.3412 | 0.3624 |
| yolo11m | 20.11 | 2.94 | **0.068220** | 0.3574 | 0.3832 |
| yolo11l | 25.37 | 3.33 | 0.062388 | 0.3707 | 0.3907 |
| yolo11x | 56.97 | 4.65 | 0.062393 | 0.3773 | 0.4087 |
| rtdetr-l | 32.97 | 7.41 | 0.065549 | 0.4578 | 0.4537 |
| rtdetr-x | 67.47 | 10.29 | **0.074507** | 0.4928 | 0.4607 |

Table 6: Performance of YOLOv5u, YOLO11, and RT-DETR on the OpenImages V7 validation set (COCO-pretrained weights, no fine-tuning, identical conditions). Bold values mark the best $\mathrm{AP}_{\mathrm{small}}$ per model family.

**Note on image resolution.** MS COCO images cluster around a 640 px long edge (most common size: $480 \times 640$, 21.5% of training images), with average of $484 \times 578$ pixels (H×W). OpenImages V7 follows a 1024 px long-edge convention (most common: $768 \times 1024$ at 25.3% of training images),

averaging $792 \times 965$ pixels–roughly $1.65\times$ larger in both dimensions. When resizing to the $640 \times 640$ input required by the Ultralytics pipeline, OpenImages undergoes a mean downsampling factor of ~$1.51\times$ along the long edge, whereas COCO's median resolution already matches the target input size, requiring no downsampling. This asymmetric scaling compresses small objects in OpenImages to a disproportionately smaller footprint on the input feature maps, contributing to the lower absolute $\mathrm{AP}_{\text{small}}$ values observed relative to COCO.

### 4.4 Cross-Dataset Performance Comparison

Table 7 summarizes the change in $\mathrm{AP}_{\mathrm{small}}$ when moving from COCO to OpenImages. **All models experience a substantial performance drop ranging between approximately 69% and 83%**. These results indicate that detectors trained on COCO do not generalize well to datasets with different object scale distributions and annotation characteristics. Interestingly, increasing model size does not consistently improve cross-dataset generalization. While larger models achieve higher accuracy on COCO, the relative performance degradation on OpenImages remains significant across most architectures.

| Model | $\Delta\mathrm{AP}_{\mathrm{small}}$ | Rel. Drop % | Latency $\Delta$ % |
|---|---|---|---|
| yolov5nu | 0.1133 | 69.4 | $-12.7$ |
| yolov5su | 0.1748 | 76.9 | $-0.2$ |
| yolov5mu | 0.2287 | 81.2 | 2.3 |
| yolov5lu | 0.2460 | 77.9 | 1.0 |
| yolov5xu | 0.2637 | 79.7 | $-3.6$ |
| yolo11n | 0.1270 | 69.5 | $-1.1$ |
| yolo11s | 0.2110 | 79.9 | $-3.0$ |
| yolo11m | 0.2461 | 78.3 | 0.5 |
| yolo11l | 0.2675 | 81.1 | $-3.1$ |
| yolo11x | 0.2906 | 82.3 | 5.3 |
| rtdetr-l | 0.2560 | 79.6 | 3.9 |
| rtdetr-x | 0.2558 | 77.4 | $-3.7$ |

Table 7: Change in $\mathrm{AP}_{\mathrm{small}}$ (COCO $\rightarrow$ OpenImages V7) and corresponding latency variation. The relative drop quantifies the domain-shift effect discussed in Sections 4.2–4.3.

### 4.5 Results Addressing the Research Questions

The research questions formulated in Section 1.2 are answered as follows:

1. **RQ1 - Which model (YOLO-based vs. DETR-based architectures) achieves the best trade-off in accuracy (mAP and $\mathrm{AP}_{\mathrm{small}}$), latency, and parameter count when evaluated on small-object-focused subsets of MS COCO and OpenImages V7?** YOLO11x achieves the highest $\mathrm{AP}_{\mathrm{small}}$ on COCO validation set (0.3527) with a latency of 4.41 ms and 57M parameters. RT-DETR-x and YOLOv5xu reach comparable accuracy (0.3303 and 0.3306) but at substantially higher computational cost. On OpenImages V7 validation set, RT-DETR-x achieves the highest $\mathrm{AP}_{\mathrm{small}}$ (0.0745), followed by YOLOv5lu (0.0700) and YOLO11m (0.0682).
2. **RQ2 - How does dataset bias, particularly the differences in small-object frequency and scale distribution between MS COCO 2017 and OpenImages V7, affect small-object detection performance (accuracy, latency, and cross-dataset generalization)?** All architectures suffer a 69-83% relative drop in $\mathrm{AP}_{\mathrm{small}}$ when evaluated on OpenImages V7 (13.39% small objects) compared with COCO (31.13% small objects).
3. **RQ3 - Does increasing model capacity (parameter count) consistently improve accuracy and the accuracy–latency trade-off for small object detection?** Larger variants consistently improve $\mathrm{AP}_{\mathrm{small}}$ on COCO (e.g., YOLO11l $\rightarrow$ YOLO11x: +0.023). However, performance gains diminish beyond ~25M parameters while latency and model size continue to increase. Notably, YOLO11s (9.46M parameters) matches or exceeds the speed of the smaller YOLO11n (2.62M parameters). Overall, architectural improvements in the YOLO11 family provide greater benefits for small-object detection than pure parameter scaling.

# 5 Discussion

Small-object detection remains inherently challenging due to feature-map downsampling and the limited spatial information available for fine-grained objects. The results of this study provide clear insights into the behaviour of modern convolutional and transformer-based detectors under small-object-focused conditions and across dataset shifts.

## 5.1 Conclusions

The research questions formulated in Section 1.2 are synthesised as follows:

**RQ1 (Model trade-off):** YOLO11x provides the best overall accuracy-latency-size balance for real-time small-object detection on COCO validation set. On OpenImages V7 validation set, RT-DETR-x shows stronger cross-dataset performance, although at a significantly higher inference cost that limits real-time applicability. Architectural design choices consistently matter more than parameter count alone.

**RQ2 (Dataset bias):** The severe cross-dataset performance drop confirms that detectors trained on COCO do not generalise well to datasets with different object-scale distributions, annotation protocols, image resolutions, and class vocabularies (80 vs 600 categories). Even though evaluation was restricted to the 24 overlapping classes, pre-training on COCO's narrower semantics still limits generalization.

**RQ3 (Model capacity and scaling):** While larger variants improve $\mathrm{AP}_{\mathrm{small}}$ on COCO, returns diminish rapidly beyond approximately 25M parameters. Architectural refinements in the YOLO11 family consistently outperform pure parameter scaling for small-object detection.

## 5.2 Limitations

Several limitations should be considered when interpreting these results. First, **all models were evaluated using pretrained weights without additional fine-tuning**. Architectures such as RT-DETR may benefit from task-specific training procedures. Second, **annotation noise and label inconsistencies** in large-scale datasets such as OpenImages **may introduce uncertainty** into evaluation metrics. Finally, **the study uses only RGB images and standard object detection pipelines**; therefore the results may not generalise to specialised domains such as monochromatic imaging.

## 5.3 Outlook

Several promising research directions could further improve small-object detection performance.

**Fine-Tuning for Cross-Dataset Robustness.** Adapting and fine-tuning object detectors on datasets with lower small-object prevalence, such as OpenImages V7, could improve robustness and reduce the observed cross-dataset performance drop. Domain adaptation techniques, including adversarial feature alignment and self-training, may further enhance generalization by mitigating distribution shifts between datasets [19].

**Scale-aware training strategies.** Recent studies propose scale-sensitive loss functions, such as Scale-Adaptive Loss (SAL) [20]. These functions assign lower weights to larger objects during training and have been shown to improve detection performance for underrepresented object scales.

**Data augmentation for small objects.** Several augmentation strategies have been proposed to improve learning of small instances [21, 6]. These include: (i) Copy-Paste Augmentation to increase the number of small instances, (ii) Mosaic Augmentation to generate multi-scale object contexts, (iii) Random Affine Transformations to simulate scale variation, (iv) MixUp Augmentation to enhance fine-grained object details.

# References

[1] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *CVPR*, 2014.

[2] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *NeurIPS*, 2015.

[3] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *CVPR*, 2016.

[4] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *ECCV*, 2020.

[5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

[6] Mate Kisantal, Zbigniew Wojna, Jakub Murawski, Jacek Naruniec, and Kyunghyun Cho. Augmentation for small object detection. In *BMVC*, 2019.

[7] Leo Thomas Ramos and Angel D. Sappa. A decade of you only look once (yolo) for object detection. *arXiv preprint arXiv:2504.18586*, 2025.

[8] Yian Zhao, Wenyu Lv, Shangliang Xu, Jinman Wei, Guanzhong Wang, Qingqing Dang, Yi Liu, and Jie Chen. DETRs Beat YOLOs on Real-time Object Detection. *arXiv preprint arXiv:2304.08069 [cs.CV]*, 2023.

[9] Pierre-Luc Asselin, Vincent Coulombe, William Guimont-Martin, and William Larrivée-Hardy. Replication study and benchmarking of real-time object detection models. *arXiv preprint arXiv:2405.06911*, 2025. Version 2, 04 Sep 2025. License: CC BY 4.0.

[10] Antonio Torralba and Alexei A. Efros. Unbiased look at dataset bias. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011.

[11] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft coco: Common objects in context. In *ECCV*, 2014.

[12] twsl and contributors. pytorchcocotools: Coco api for pytorch. `https://github.com/twsl/pytorchcocotools`, 2020.

[13] Alina Kuznetsova, Hassan Rom, Neil Alldrin, Jasper Uijlings, Ivan Krasin, Jordi Pont-Tuset, Shahab Kamali, Stefan Popov, Matteo Malloci, Alexander Knapic, et al. The open images dataset v4. *International Journal of Computer Vision*, 128:1956–1981, 2020.

[14] Glenn Jocher. Ultralytics yolov5, 2020.

[15] Glenn Jocher and Jing Qiu. Ultralytics yolo11, 2024.

[16] Lihao Liu et al. Ao Wang, Hui Chen. Yolov10: Real-time end-to-end object detection. *arXiv preprint arXiv:2405.14458*, 2024.

[17] Ultralytics. Rt-detr models — ultralytics documentation. `https://docs.ultralytics.com/models/rtdetr/`, 2024.

[18] PyTorch Contributors. PyTorch Documentation: Benchmarking. `https://pytorch.org/docs/stable/benchmark_utils.html`, 2025.

[19] Yuhua Chen, Wen Li, Christos Sakaridis, Dengxin Dai, and Luc Van Gool. Domain adaptive faster r-cnn for object detection in the wild. In *CVPR*, 2018.

[20] Zhen Liu, Yifan Wang, Hao Chen, and Ming Li. Scale-aware relay and scale-adaptive loss for tiny object detection. *arXiv preprint arXiv:2511.09891*, 2025. Proposes SARL (Scale-Aware Relay Layer) and SAL (Scale-Adaptive Loss).

[21] Ultralytics. Yolov5 documentation: Model architecture and data augmentation. `https://docs.ultralytics.com/yolov5/tutorials/architecture_description/`, 2023.